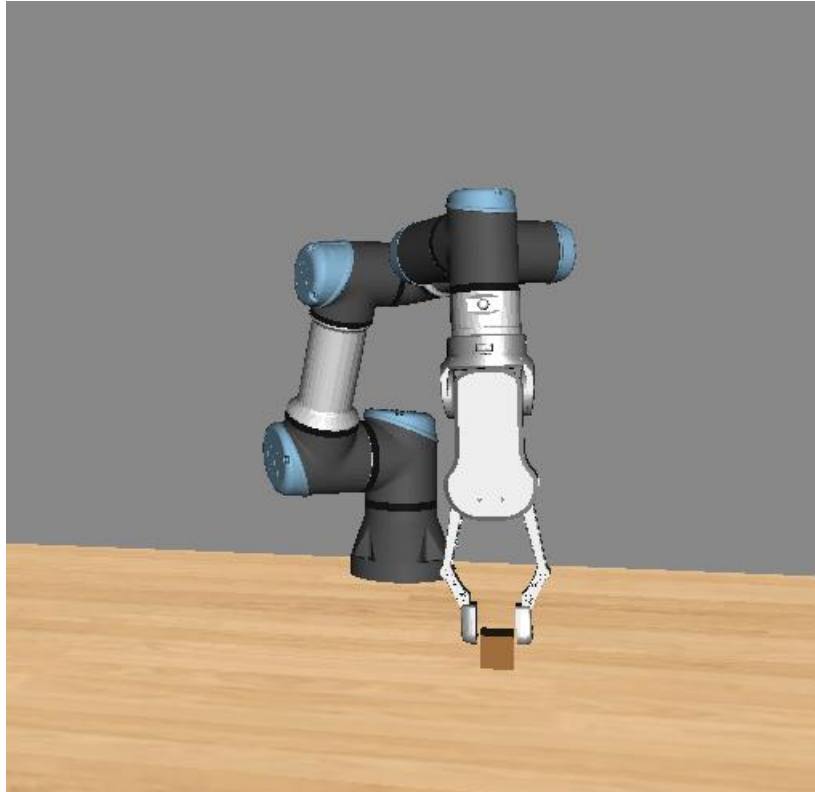


Proyecto Final:

Parte de manipuladores robóticos



Información

- El peso de esta parte del proyecto final en la nota final de la asignatura es de un 15% (50% del 30%).
- La evaluación será mediante una demo final durante el mes de junio (en hora-rio a convenir con los estudiantes). Dicha demo incluirá un turno de preguntas en las que los profesores pueden preguntar a cualquier miembro del grupo so-bre cualquier aspecto de la propuesta hecha por los estudiantes.
- El mismo día de la evaluación habrá que entregar todo el código (y, opcional-mente videos de la demo) a través de Atenea.
- A modo orientativo, si P es la nota total de la parte de manipuladores robóticos del proyecto final, $P = 0.6 * D + 0.4 * TP$, donde D es la nota de la demo real (y en caso de que falle, de los videos y código subidos el mismo día) y TP es la nota de cada alumno en el turno de preguntas.
- Si solo se resuelve el proyecto final en simulación, es decir, sin usar el robot real, la nota máxima que se puede obtener en esta parte del proyecto final es un 7, donde 4.2 puntos corresponderán a la demo simulada y 2.8 a la nota asignada a cada alumno en el turno de preguntas.

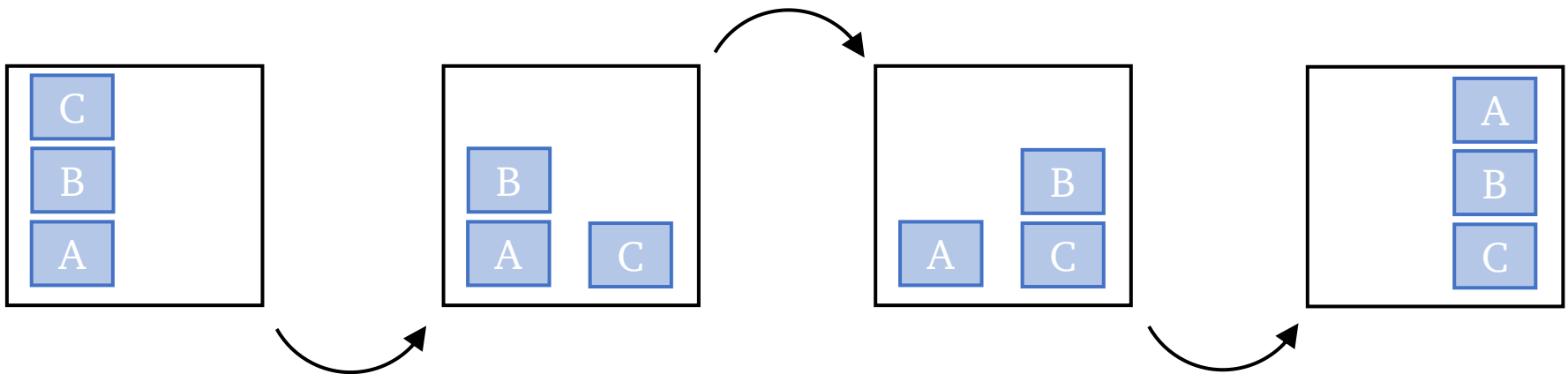
Información

El proyecto final consiste en programar un robot manipulador para ejecute una cierta tarea de manipulación. Los pasos a seguir son los siguientes:

1. Escoger una tarea de manipulación que involucre al menos dos secuencias de *pick-move-place*. Se premiara especialmente problemas con sentido, escenarios no triviales y se-cuencias *pick-move-place* extra.
2. Se ha de formular adecuadamente la tarea de manipulación, así como los estados inicial y final. A partir de ellos, se debe crear los *domain* y *problem files* (PDDL) asociados a la tarea de manipulación escogida. Mostrar dichos ficheros PDDL.
3. Resolver el problema de planificación de tareas asociado al problema de manipulación escogido y mostrar la secuencia de acciones necesaria para resolverlo tal y como se calcula usando el servicio de ROS 2 `downward_service`.
4. Resolver el problema de planificación de tareas y movimientos (TAMP por sus siglas en inglés) asociado al problema de manipulación escogido y mostrar el `taskfile_tamp-config_name` obtenido. Mostrar la simulación de la resolución obtenida en simulación usando *The Kautham Project*.
5. Implementar y ejecutar la solución obtenida en el robot real (UR3e).

Ejemplo

Resolver el problema de las torres de Hanoi simplificado con tres piezas (tres secuencias de *pick-move-place*, un problema real en planificación robótica y con estado inicial (primera figura a la izquierda) y estado final (última figura a la derecha).



Instrucciones extra

1. **Obstáculos y objetos a manipular** → En los ficheros de *The Kautham Project*, los obstáculos (lo que incluye a los objetos con los que hay que hacer *pick-and-place*) vienen descritos con una posición y orientación en el siguiente formato (TH, WZ, WY, WX, Z, Y, X), mientras que en los ficheros *tampconfig* vienen en formato (X, Y, Z, WX, WY, WZ, TH) o (X, Y, Z, TH, WX, WY, WZ). Este formato es, para la parte de orientación, un eje-ángulo (como un cuaternión), es decir, un eje de rotación, con vector director (WX, WY, WZ) y un ángulo de rotación con valor TH. Cuando definais la posición de un objeto o un obstaculo, hacerlo usando una orientación sencilla y representarla usando ángulos de Euler (porque es fácil e intuitivo). Luego, usad las formulas que podéis encontrar en la bibliografía de la asignatura (Siciliano et al. *Robotics modelling, planning and control*, capítulo 2) o en internet para pasar dicha orientación de los objetos en ángulos de Euler a eje-ángulo. Lo podéis hacer de otras formas que se os ocurran, siempre que terminéis calculando la orientación en formato eje-ángulo para poder ponerla en los ficheros de Kautham y/o en los ficheros *tampconfig*.

Instrucciones extra

2. **Fichero `tampconfig`** → Tal y como vimos en la sesión de prácticas P4 (donde podéis encontrar más información, para poder hacer correctamente el TAMP, hay que modificar el fichero `tampconfig`. Allí encontrareis:
- a) Una primera parte, llamada `Problemfiles`, donde se llama al planificador de movimientos, los ficheros PDDLs, etc.
 - b) Una segunda parte, llamada `States`, donde se incluyen los distintos objetos y robots que intervendrán en la manipulación. Allí se define también la posición y orientación de los objetos y, en el caso del robot, su configuración como control (es decir, como vector en escala cero-uno). Dichas posiciones, orientaciones y configuraciones iniciales se ha de cambiar también en el *problem file* de *Kautham*, `OMPL_RRT_...`
 - c) Una tercera parte, llamada `Actions`, donde se incluyen las dos acciones, *pick*, y *place*, que el robot puede ejecutar. Ahí encontrareis dos tipos de control, `HomeControls` y `GraspControls`. Los primeros se usan para definir la configuración inicial o *home* del robot para dicha acción (para un *place*, dicha configuración es la configuración a la que el robot va después de soltar el objeto). Los segundos, por otro lado, se utilizan para definir las configuraciones en las que, cerrando o abriendo la pinza, se ejecutan las acciones *pick* y *place*. Como ambas son variables de tipo control, solo aceptan números entre cero y uno.

Instrucciones extra

¿Cómo encontrar estos números? Hay dos maneras:

1. Coger una configuración del UR3e real, pasarla a radianes, y normalizarla como sigue:

1. Para las articulaciones 1, 2, 4, 5 y 6 (todas menos el codo):

$$q_{i\text{normalizada}} = \frac{q_i + 2\pi}{4\pi}$$

2. Para la articulación 3 (el codo):

$$q_{i\text{normalizada}} = \frac{q_i + \pi}{2\pi}$$

2. Abrir el *Kautham*, ir a las barras laterales que hay a la izquierda de la pantalla principal, encontrar la barra de controles, y con los botones encontrar una configuración inicial, una final, una de *pick*, y una de *place*. Se pueden copiar y pegar estos valores, ya en formato control (es decir, entre cero y uno) en la parte correspondiente del fichero `tampconfig`.